📘 Microservices Communication — Notes

🎯 Overview

> Microservices communication is how independent services interact to complete a business workflow.

🔗 Inter-Service Communication

Microservices communicate with each other to complete end-to-end workflows.
This communication can be broadly classified into:

- **Synchronous Communication**
- **Asynchronous Communication**

🔵 Synchronous Communication

In synchronous communication, one service calls another service and **waits for a response** before continuing.

📌 Characteristics

• Blocking communication
• Request-response model
• Sequential flow of execution

✅ Advantages

• Simple and easy to understand
• Immediate response
• Easier debugging and tracing
• Strong consistency across services

❌ Disadvantages

• Tight coupling between services
• High latency due to multiple calls
• Cascading failures (one failure affects entire flow)

💻 Examples

• RestTemplate
• WebClient
• OpenFeign

🟢 Asynchronous Communication

In asynchronous communication, services communicate using **events or messages**, without waiting for an immediate response.

📌 Characteristics

• Event-driven communication
• Non-blocking (no waiting)
• Loose coupling between services
• Parallel execution of services
• Eventual consistency (not immediate)
• Independent processing by each service

✅ Advantages

• High scalability
• Loose coupling between services
• Better fault tolerance
• Faster user response (no blocking)

• Supports event-driven architecture
• Enables parallel processing

❌ Disadvantages

• Harder to debug and trace
• Eventual consistency (delayed updates)
• More complex architecture
• Handling duplicate events and ordering issues
• Increased operational overhead (queues, brokers, monitoring)

💻 Examples

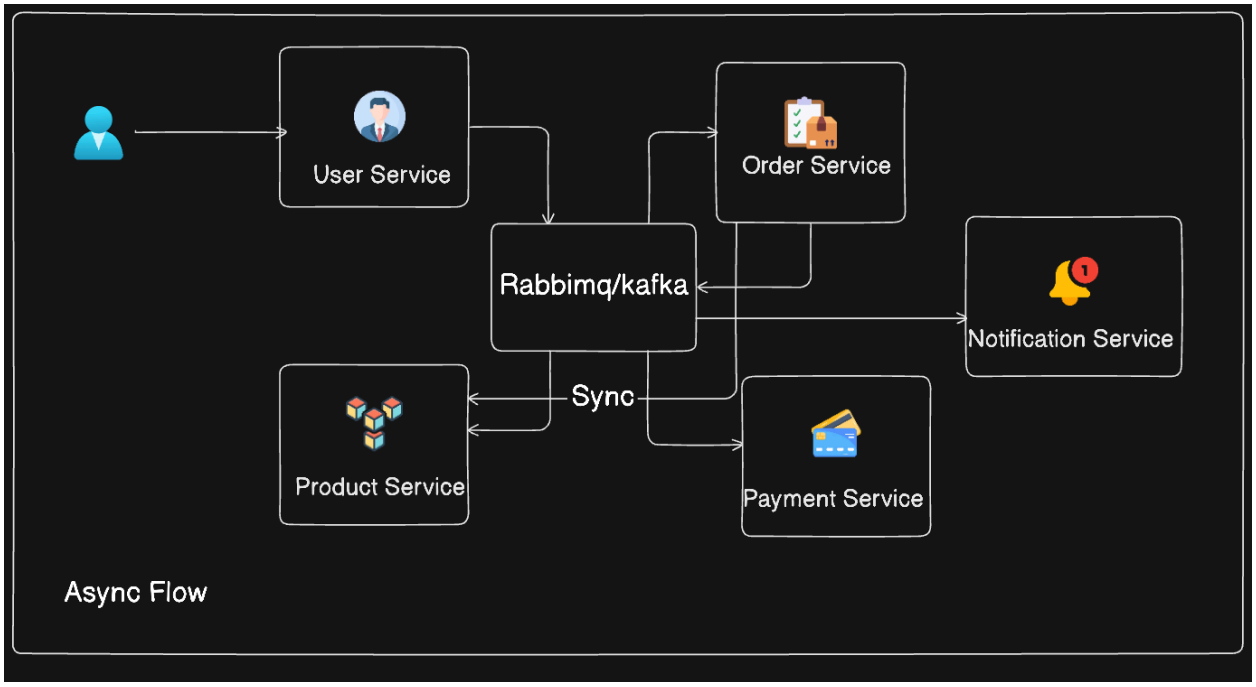
• RabbitMQ
• Apache Kafka

⚖️ Summary

Synchronous → Simple, immediate, but tightly coupled
Asynchronous → Scalable, decoupled, but more complex

🎯 Key Takeaway

> In real-world systems, both synchronous and asynchronous communication are used together — synchronous for immediate decisions, and asynchronous for background processing.


User Service
Order Service
Rabbimq/kafka
Notification Service
Sync
Product Service
Payment Service
Async Flow